Contents

Guide 4 — The Class Diagram (the data model), explained simply

What the class diagram in Chapter 2 means, in plain words: what each box is, what the arrows mean, and a small glossary at the end (including what “slug” means).

---

1. What the diagram is

The class diagram is just **the list of things the app stores**, drawn as boxes. Each box is one kind of record (one table in the database). A box has **two parts**, top to bottom:

1. **Name** — the kind of thing (Operator, Offer, User…).
2. **Attributes** — the pieces of data it keeps, named exactly as in the database (an Offer keeps `name`, `priceDA`, `dataGB`, `validityDays`, …).

There is deliberately **no “methods” part**. This is a *data* model — the database tables — so the boxes hold data, not behaviour. The behaviour (filtering offers, ranking, login checks, sending notifications…) lives in separate functions in the code (`src/lib`), not on these records, so putting methods on the boxes would be inaccurate. The **arrows** between boxes show how the records are linked.

---

2. The eight boxes, one line each

- **Operator** — a mobile operator (Djezzy, Ooredoo, Mobilis). Stores its `name`, `websiteUrl`, `logoUrl`, and brand `primaryColor`.
- **Offer** — one mobile plan. Stores `name`, `type` (prepaid / postpaid / data only), `priceDA`, `dataGB`, `voiceMinutes`, `smsCount`, `validityDays`, `network`, `features`, and `isActive`.
- **PriceHistory** — one row per price change of an offer (so the price over time can be shown). Stores `priceDA` and `recordedAt`.
- **User** — a registered account. Stores `name`, `email`, `passwordHash`, and `role` (user or admin).
- **UserProfile** — the user’s stated needs for the recommendation engine. Stores `monthlyBudget`, `dataUsageGB`, `voiceMinutes`, `smsCount`, the preferences, and the `priorities` ranking.
- **SavedOffer** — a bookmark linking one user to one offer they saved. Stores `savedAt`.
- **Notification** — one alert for a user (new offer, price drop, recommendation, or an admin alert). Stores `title`, `message`, `type`, and `isRead`.
- **ScrapeLog** — one record per scrape run, for the admin dashboard. Stores `status`, the offers `found` / `added` / `updated` / `deactivated` counts, any `errorMessage`, and the `duration`.

---

3. The arrows (relationships) and the "1" and "N"

An arrow is a **link** between two records. The small **1** and **N** say *how many*:

- **Operator 1 — N Offer**: one operator has **many** offers; each offer belongs to **one** operator.
- **Operator 1 — N ScrapeLog**: one operator, **many** scrape runs over time.
- **Offer 1 — N PriceHistory**: one offer, **many** price records over time.
- **User 1 — N Notification**: one user, **many** notifications.
- **User 1 — 1 UserProfile**: each user has **exactly one** profile.
- **User 1 — N SavedOffer** and **Offer 1 — N SavedOffer**: SavedOffer sits in the middle and links the two, so **a user can save many offers and an offer can be saved by many users** (this is the standard way to record a "many to many" link).

So, read aloud: *"one operator has many offers; each offer keeps a history of prices; a user has one profile, many notifications, and many saved offers."*

---

4. Small glossary (the words that look technical)

- **Attribute** — a piece of data a record stores (an Offer's *price*).
- **Behaviour** — what the app *does* with these records (filter, rank, save, notify). It is **not** drawn on this diagram: a data model only describes stored data. The behaviour lives in separate functions in `src/lib` (e.g. `recommendOffers`, `validateOffer`, `searchOffersFts`).
- **Association** — the arrow; a link between two records.
- **1 and N** — how many on each side. **1 — N** = "one to many"; **1 — 1** = "one to one".
- **slug** — a short, web-safe version of a name: lowercase, with dashes instead of spaces and accents removed. For example the offer "Djezzy LEGEND 4000" has the slug `djezzy-legend-4000`. It is used inside web addresses and to match a freshly scraped offer back to its existing database row. It is a technical detail the user never sees, which is why it is **left out of the simplified diagram**.
- **-1 means unlimited, 0 means none** — a convention for calls, SMS, and data, so the app can tell "unlimited" apart from "zero".
- **Primary key (PK)** — the **underlined** `id` at the top of every box: a unique value (a cuid) that identifies one record. Every entity has one. The diagram marks it by underlining the `id` (the standard UML convention) instead of writing a "(PK)" tag.
- **Foreign key (FK)** — how one record points to another's `id`. These are exactly the **arrows**: e.g. the Operator → Offer arrow is the `operatorId` foreign key stored on each Offer; SavedOffer holds `userId` and `offerId`. So the keys are not missing — the PK is the `id` row, and every FK is an arrow.
- **Unique key** — a field whose value cannot repeat: a User's `email`, an Operator's `name` and `slug`.

---

5. Jury questions — ready answers

- **Why eight boxes?** Each is one real responsibility: the catalogue (Operator, Offer, PriceHistory, ScrapeLog) and the user side (User, UserProfile, SavedOffer, Notification).
- **Why is UserProfile separate from User?** A user may exist without ever filling a recommendation profile; keeping the profile in its own box keeps the account simple and lets the profile change without touching the account.
- **Why is SavedOffer its own box and not just a list on the user?** Because the same offer can be saved by many users and a user can save many offers; a small linking record is the clean way to store that.

- **Why are there no methods on the boxes?** Because it is a *data* model — these are the database tables, which store data, not behaviour. In this Next.js + Prisma stack the models are plain records with no methods of their own; the behaviour is implemented as separate functions in `src/lib` (for example `recommendOffers`, `validateOffer`, `searchOffersFts`, `scrapeDjezzy`). Drawing those as class methods would be inaccurate, so the diagram correctly shows attributes and relationships only — matching the Prisma schema exactly.
- **What does the underlined `id` mean / where are the keys?** The **underlined** `id` at the top of every box is the **primary key** — the unique identifier each record has (that's why every table has one). The **foreign keys** are the arrows: each association (Operator→Offer, Offer→PriceHistory, User→SavedOffer→Offer, …) is stored as a foreign-key column (`operatorId`, `offerId`, `userId`). Table 2.5 lists each FK next to its relationship.
- **Where is the full detail (every field, every type)?** In the entities table (Table 2.5) next to the diagram in Chapter 2; the diagram is the simple overview.

Companion to the recommendation, scraping, and search guides. Tell me the next concept you want explained.